

실전 웹서비스개발 - 3주차

React 프레임워크를 이용한 웹 프론트엔드 개발

학습 목표

- React의 등장 배경과 핵심 개념 이해
- Component 기반 사고와 State 관리
- 함수형 컴포넌트 개발 방법
- React 생태계를 활용한 개발 방법
- 서버리스 환경에서 React 웹 서비스 배포

React Framework

React

2013년 5월 29일 Facebook(현 Meta)에서 발표한 프론트엔드 개발 프레임워크

- 초기 Facebook(현 Meta)이 개발, 현재는 Meta + 커뮤니티가 유지관리
- 2017-09-22 Meta가 MIT 라이선스로 재라이선스 발표(React 16부터 적용)
- 2025-08-28일 현재 v19.1.1
- <https://react.dev/>
- <https://github.com/facebook/react>



실무 사용률 순위 (Stack Overflow 2025, 전문 개발자 응답)

1. **React 46.9%**
2. jQuery 24.1% (레거시 유지보수 비중 ↑)
3. Next.js 21.5%
4. Angular 19.8%
5. Vue.js 18.4%
6. Svelte 6.9%

<https://survey.stackoverflow.co/2025/technology>

React 등장 배경

웹 개발의 발전 과정

- 1세대: HTML + CSS (정적 웹페이지)
- 2세대: jQuery 시대 (DOM 직접 조작)
- 3세대: SPA 프레임워크 등장 (Angular, React, Vue)

jQuery 문제점

- jQuery 중심의 명령형 DOM 조작과 분산된 이벤트/상태 관리가 대규모 동적 UI에서 유지보수·성능·일관성 문제를 초래.

React의 해결 방식

- 선언적 프로그래밍: "화면이 어떻게 보여야 하는지"만 선언
- 자동 동기화: 데이터가 바뀌면 화면도 자동으로 업데이트
- 예측 가능성: 단방향 데이터 흐름으로 디버깅 용이

명령형 프로그래밍(jQuery) VS 선언적 프로그래밍(React)

❌ 명령형 (Imperative)

```
// "어떻게" 할 것인지 단계별로 명시
const button = document.createElement('button');
button.innerText = 'Click me';
button.className = 'btn-primary';
button.addEventListener('click', () => {
  const count = parseInt(button.innerText) + 1;
  button.innerText = `Clicked ${count} times`;
});
document.body.appendChild(button);
```

✅ 선언적 (Declarative)

```
// "무엇"을 원하는지 선언
function Button() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Virtual DOM

메모리 안에 존재하는 가벼운 UI 트리(자바스크립트 객체)

- 실제 DOM 조작은 무겁다
- 상태 → UI 선언 결과를 실제 DOM에 최소 변경으로 반영
- 전체를 다시 그리는 것과 동일한 결과 실제로는 차이만 커밋



Render는 마음껏, Commit은 최소화

DOM 직접 조작 방식 vs Virtual DOM 사용

Real DOM 조작(jQuery)

- DOM 조작은 비용이 큼
- 불필요한 전체 렌더링
- 레이아웃 재계산 빈번
- 성능 저하 발생

Virtual DOM 사용(React)

- JavaScript 객체로 표현
- 변경된 부분만 감지
- 최소한의 DOM 조작
- 배치 업데이트로 최적화

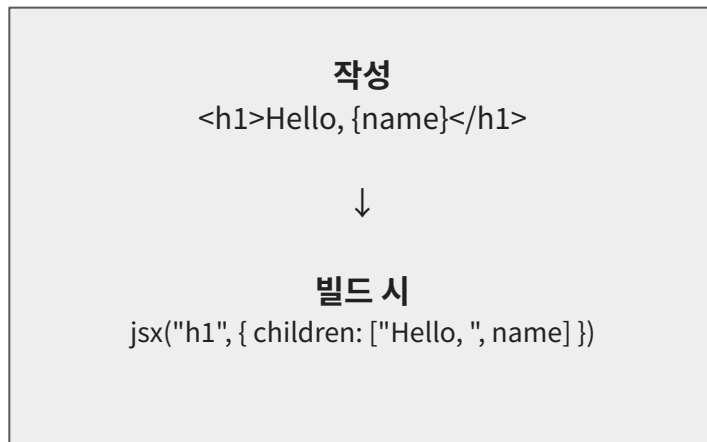
JSX(JavaScript XML)

JavaScript 안에서 UI 트리를 “선언”하는 문법 확장

- HTML처럼 보이지만 문자열이 아니라 JS 표현식
- 빌드 시 함수 호출(jsx/jsxs) 로 변환
- 런타임에 DOM을 직접 만들지 않고, 가상 노드(VDOM) 를 만들도록 변환

목적

- 상태 → UI를 읽기 쉬운 문법으로 기술



JSX(JavaScript XML)

JSX 작성 규칙

- 하나의 루트 요소로 감싸야 함

<>...</>(Fragment)

- 모든 태그는 닫혀야 함 (self-closing 포함)
- JavaScript 표현식은 {} 안에 작성
- 속성 이름은 camelCase

(onclick❌ → onClick✅)

- 주석

{/* 주석 */}

JSX 작성 예

```
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <GlobalStyles />
        <div className="app">
          <Header />
          <main className="main-content">
            <Routes>
              <Route path="/" element={<HomePage />} />
              <Route path="/list" element={<ListPage />} />
              <Route path="/restaurant/:id" element={<DetailPage />} />
              <Route path="/popular" element={<PopularPage />} />
              <Route path="/submit" element={<SubmitPage />} />
              <Route path="*" element={<NotFound />} />
            </Routes>
          </main>
          <footer className="footer">
            <p>© 2025 Ajou Campus Foodmap | Made with React</p>
          </footer>
        </div>
        <ToastContainer
          position="bottom-right"
          autoClose={3000}
          hideProgressBar={false}
          newestOnTop={false}
          closeOnClick
          rtl={false}
          pauseOnFocusLoss
          draggable
          pauseOnHover
          theme="light"
        />
      </BrowserRouter>
    </QueryClientProvider>
  );
}
```

Component

입력 또는 내부 상태 정보에 따라 UI를 반환하는 재사용 가능한 UI 단위

- UI = (props, state) → 화면
- DOM을 직접 조작하지 않고 선언적으로 “보이고 싶은 모습”만 기술

설계 배경(jQuery 사용 시 어려움)

- 여러 코드가 같은 DOM을 건드리며 상태/화면 불일치, 유지보수 난이도 상승
- 대규모 동적 UI(피드, 코멘트, 알림): “어디서 무엇을 바꿨는지” 추적이 어려움

목적

- 화면을 작게 쪼개어 조합
- 화면 갱신은 상태 변경 → 자동 렌더

Component

Component 설계 원칙

- **단일 책임 원칙:** 하나의 Component는 하나의 기능만
- **재사용성:** 다양한 곳에서 활용 가능하도록 설계
- **조합 가능성:** 작은 Component를 조합해 복잡한 UI 구성할 수 있게

Component 분리 기준

- 재사용이 필요한 경우
- 로직이 복잡해진 경우
- 성능 최적화가 필요한 경우
- 테스트를 분리하고 싶은 경우

Component 구현 방식

클래스형(Class, 레거시&에러 바운더리): render() + 라이프사이클 메서드

```
import React from "react";

export class CounterClass extends React.Component {
  state = { n: 0 };
  inc = () => this.setState(s => ({ n: s.n + 1 }));

  componentDidMount() { document.title = `n=${this.state.n}`; }
  componentDidUpdate() { document.title = `n=${this.state.n}`; }

  render() {
    return <button onClick={this.inc}>Count: {this.state.n}</button>;
  }
}
```

함수형(Function, 권장 표준): JSX 반환 + Hooks로 상태/효과/컨텍스트 처리

```
import { useEffect, useState } from "react";

export function CounterFn() {
  const [n, setN] = useState(0);
  useEffect(() => { document.title = `n=${n}`; }, [n]);

  return <button onClick={() => setN(s => s + 1)}>Count: {n}</button>;
}
```

상태(State)

렌더링 결과에 영향을 주는, 시간에 따라 변하는 값

- ‘현재 화면이 무엇처럼 보여야 하는가’를 결정하는 최소 정보
- UI는 상태의 함수 → 상태가 바뀌면 화면이 다시 그려짐

구분

- **Props**: 외부에서 들어오는 읽기 전용 값
- **State**: 컴포넌트 내부에서 바뀌는 값
- **Derived**: props/state로 즉시 계산 가능한 값(별도 저장 X)

컴포넌트 구조

```
import { useState } from "react";

export default function ProfileCard({ name = "익명", skills = [], children }) {
  // 1) State: 내부에서 변하는 값
  const [liked, setLiked] = useState(false);

  // 2) Derived(파생값): state로부터 즉시 계산 (중복 저장 X)
  const skillCount = skills.length;

  // 3) Event Handler: UI와 상호작용 연결
  const toggleLike = () => setLiked(v => !v);

  // 4) Render(JSX): 조건부/목록/children 포함
  return (
    <section className={`card ${liked ? "liked" : ""}`}>
      <header>
        <h3>{name}</h3>
        <button onClick={toggleLike}>{liked ? "★ Liked" : "☆ Like"}</button>
      </header>

      {/* 목록 + key */}
      <ul>
        {skills.map(s => (
          <li key={s}>{s}</li>
        ))}
      </ul>

      {/* 조건부 렌더링 + 파생값 표시 */}
      <p>{skillCount === 0 ? "No skills yet" : `${skillCount} skills`}</p>

      {/* Composition: 부모가 넣는 추가 UI */}
      <footer>{children}</footer>
    </section>
  );
}
```

이벤트(Event)

사용자의 행동(클릭/입력/제출/키보드/포커스 등)이 발생할 경우 반응하는 콜백 함수

- 상태를 변경 → 렌더 재실행
- 사용자·DOM 상호작용 트리거
- JSX 속성으로 등록: **onClick**, **onChange**, **onSubmit**, ...

컴포넌트 구조

```
import { useState } from "react";

export default function ProfileCard({ name = "익명", skills = [], children }) {
  // 1) State: 내부에서 변하는 값
  const [liked, setLiked] = useState(false);

  // 2) Derived(파생값): state로부터 즉시 계산 (중복 저장 X)
  const skillCount = skills.length;

  // 3) Event Handler: UI와 상호작용 연결
  const toggleLike = () => setLiked(v => !v);

  // 4) Render(JSX): 조건부/목록/children 포함
  return (
    <section className={`card ${liked ? "liked" : ""}`}>
      <header>
        <h3>{name}</h3>
        <button onClick={toggleLike}>{liked ? "★ Liked" : "☆ Like"}</button>
      </header>

      { /* 목록 + key */ }
      <ul>
        {skills.map(s => (
          <li key={s}>{s}</li>
        ))}
      </ul>

      { /* 조건부 렌더링 + 파생값 표시 */ }
      <p>{skillCount === 0 ? "No skills yet" : `${skillCount} skills`}</p>

      { /* Composition: 부모가 넣는 추가 UI */ }
      <footer>{children}</footer> //
    </section>
  );
}
```

훅(Hook)

상태/부수효과/컨텍스트/참조 등을 다루게 해 주는 함수 API 집합

- “UI = State의 함수”를 실천
- 커스텀 훅으로 로직을 재사용 가능

도입 배경

- 클래스형 컴포넌트의 라이프사이클 분산 문제
- 컴포넌트의 재 사용 시 High-Order Component/Render Props 남용 문제
- stateful 로직을 함수 단위로 깔끔하게 재사용

2019 (React 16.8): Hooks 정식 출시

- 현재 함수형 컴포넌트가 표준

- **상태/효과**: useState, useEffect
- **참조/레이아웃**: useRef, useEffect
- **공유/전이**: useContext, useReducer
- **성능 최적화(필요할 때만)**: useMemo, useCallback
- **동시성·UX(선택)**: useTransition, useDeferredValue

Hook 종류

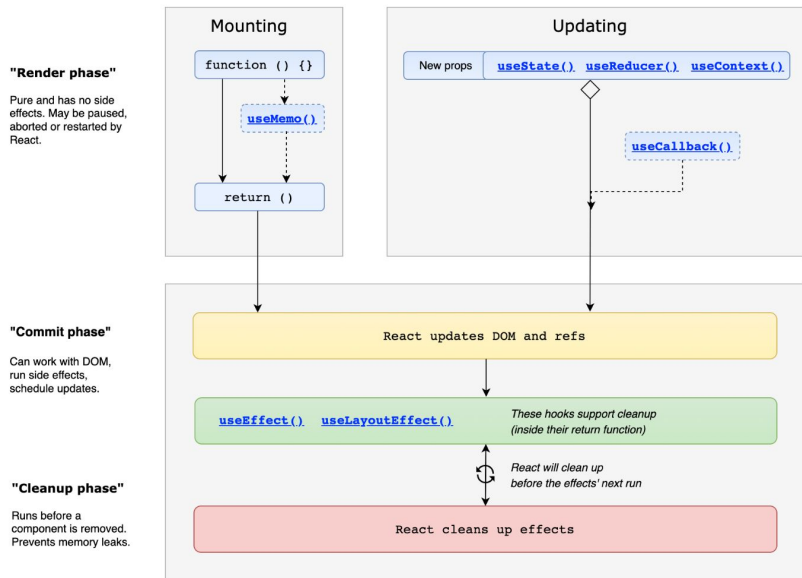
```
import { useEffect, useState } from "react";

export function CounterFn() {
  const [n, setN] = useState(0);
  useEffect(() => { document.title = `n=${n}`; }, [n]);

  return <button onClick={() => setN(s => s + 1)}>Count: {n}</button>;
}
```

함수형 컴포넌트 라이프사이클(LifeCycle)

컴포넌트가 생성되는 시점부터 화면에 나타나고, 사라질 때 발생하는 이벤트 순서



<https://wavez.github.io/react-hooks-lifecycle/>

```
function Demo({ list }) {
  const [n, setN] = useState(0); // Updating 트리거
  const total = useMemo(() => list.length + n, [list, n]); // 렌더 중 계산
  const onClick = useCallback(() => setN(v => v + 1), []); // 렌더 중 함수 메모

  useLayoutEffect(() => {
    // Commit 직후(페인트 전) DOM 측정/스크롤 조정
    document.getElementById('btn')?.scrollIntoView({ block: 'nearest' });
  }, [total]);

  useEffect(() => {
    // 페인트 후: 문서 제목/타이머/구독/네트워크 등
    document.title = `total=${total}`;
    return () => console.log('cleanup before next effect or unmount');
  }, [total]);

  return <button id="btn" onClick={onClick}>Total: {total}</button>;
}
```

setN → Render(useMemo/useCallback 계산) → Commit(DOM 패치) → useLayoutEffect → 페인트 → useEffect → 다음 업데이트 전 cleanup

React 생태계

React 코어만으로는 부족한 작업을 해결해 주는 라이브러리·도구의 묶음과 커뮤니티

- 작은 코어 + 조립 철학: “원하는 걸 선택/조합하라”
- NPM 네트워크 효과: 수많은 패키지·플러그인·템플릿이 빠르게 축적
- 다양한 런타임: 브라우저, 서버, 모바일, 데스크톱
- 커뮤니티 규모: 사례·스타일가이드·UI 키트·도구가 폭발적으로 늘어남

주요 패키지

- react: 컴포넌트·Hooks로 UI를 선언적으로 만드는 핵심 라이브러리.
- react-dom: React UI를 브라우저 DOM에 렌더하는 바인딩.
- Vite: ESM 기반 초고속 개발 서버, 빌드 시 Rollup
- react-hook-form: 고성능 폼 상태/검증 관리(비제어 입력 기반).
- react-router-dom: 브라우저용 라우팅(URL↔화면, 중첩/동적 경로).
- @emotion/react: React에서 css prop/Theme를 사용하는 CSS-in-JS 런타임.
- @tanstack/react-query: 서버 상태(fetch/캐시/동기화/무효화/재시도) 관리 표준.
- axios: Promise 기반 HTTP 클라이언트(요청/응답 인터셉터 등).

package.json

```
{
  "name": "pwd-week3",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@emotion/react": "^11.14.0",
    "@emotion/styled": "^11.14.1",
    "@tanstack/react-query": "^5.87.4",
    "@testing-library/dom": "^10.4.1",
    "@testing-library/jest-dom": "^6.8.0",
    "@testing-library/react": "^16.3.0",
    "@testing-library/user-event": "^14.6.1",
    "axios": "^1.12.1",
    "react": "^19.1.1",
    "react-dom": "^19.1.1",
    "react-hook-form": "^7.62.0",
    "react-icons": "^5.5.0",
    "react-router-dom": "^7.9.1",
    "react-spinners": "^0.17.0",
    "react-toastify": "^11.0.5",
    "web-vitals": "^5.1.0"
  },
}
```

Netlify 소개

Netlify

Git 기반 워크플로로 웹앱/동적 사이트를 빌드·배포하고, 서버리스/백엔드를 제공하는 클라우드 개발 플랫폼

- 정적 파일을 CDN으로 제공
- 다양한 플러그인·API 통합 지원
- **JAMstack** 아키텍처를 개척
 - **JAM = JavaScript · APIs · Markup**
 - 프론트엔드는 사전 생성된 정적 마크업(SSG) 을 CDN에서 제공
 - 동적 기능은 클라이언트 JS가 외부 API/서버리스 함수로 호출해 구현



서버리스(Serverless) 아키텍처

서버 프로비저닝/패치/스케일링은 클라우드가 맡고, 함수 단위 코드 를 배포해 호출한 만큼만 과금하는 아키텍처

- **FaaS(Function as a Service):** 요청이 올 때 함수(handler) 실행 → 응답 후 리소스 정리
- **Edge Functions:** 전 세계 엣지에서 요청 가로채 초저지연
- **BaaS(Backend as a Service):** 인증/DB/스토리지 등 백엔드 기능을 서비스로 사용
- **빌드-배포 플로우:** Git push → CI가 빌드 → 함수/정적 파일을 CDN + 실행 환경에 배포

장점

- **운영 부담 감소:** 서버 관리/패치/오토스케일링 자동
- **비용 효율:** 사용한 만큼 과금, 유휴 비용 거의 없음
- **민첩성:** 작은 함수로 빠르게 기능 추가

주의점

- **콜드 스타트:** 처음/유휴 후 첫 요청 지연 가능
- **실행 제한:** 시간/메모리/파일 시스템 제약
- **DB 커넥션:** 매 요청 열지 말고 글로벌에 재사용 캐시
- **벤더 종속성:** 플랫폼별 빌드/런타임 차이 고려

Netlify Forms

별도 서버 없이도 폼 제출을 수집·저장·알림까지 해주는 Netlify의 내장 기능

- Netlify가 배포 시 HTML을 크롤링해 <form>을 등록하고, 제출 데이터를 대시보드에 저장
- <form name="...">에 netlify 속성이 있으면 등록 대상.
- SPA/React 지원: 빌드 타임에 실제 마크업이 없을 수 있으므로 히든 필드 input[name="form-name"] 로 폼 이름을 명시.
- 스팸 방지: honeypot(netlify-honeypot="bot-field") + reCAPTCHA 연동.
- 알림/연동: 이메일, Slack, Webhook, Zapier 등. Submissions API로도 접근 가능.
- 고급: Functions와 연결해 submission-created 이벤트로 후처리(예: Slack 알림, DB 저장).

Netlify Forms 등록

Netlify Forms 등록(public/index.html)

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width,
initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta name="description" content="캠퍼스 주변 맛집을 찾아보세요!" />
    <title>Ajou Campus Foodmap - 우리 학교 맛집 지도</title>
  </head>
  <body>
    <noscript>이 앱을 실행하려면 JavaScript를 활성화해야
합니다.</noscript>

    <!-- Netlify Forms를 위한 숨겨진 폼 -->
    <form name="restaurant-submit" netlify hidden>
      <input type="text" name="restaurantName" />
      <select name="category"></select>
      <input type="text" name="location" />
      <input type="text" name="priceRange" />
      <textarea name="recommendedMenu"></textarea>
      <textarea name="review"></textarea>
      <input type="text" name="submitterName" />
      <input type="email" name="submitterEmail" />
    </form>

    <div id="root"></div>
  </body>
```

Netlify Forms 등록(src/components/SubmitRestaurant.js)

```
function SubmitRestaurant() {
  const [submitted, setSubmitted] = useState(false);
  const { register, handleSubmit, formState: { errors, isSubmitting },
reset } = useForm();

  const onSubmit = async (data) => {
    try {
      // Netlify Forms로 제출
      const response = await fetch("/", {
        method: "POST",
        headers: { "Content-Type": "application/x-www-form-urlencoded"
},
        body: new URLSearchParams({
          "form-name": "restaurant-submit",
          ...data
        }).toString()
      });

      if (response.ok) {
        setSubmitted(true);
        toast.success('맛집이 성공적으로 제보되었습니다! 🍽️');
        reset();
        setTimeout(() => setSubmitted(false), 5000);
      } catch (error) {
        toast.error('제출 중 오류가 발생했습니다.');
```

Netlify Forms



- Project overview
- Project configuration
- Deploys
- Preview Servers
- Logs
- Metrics
- Domain management
- Forms**
- Blobs

Projects / ajou-campus-foodmap

News Support

Submissions from restaurant-submit

Last submission on Sep 14 (16 hours ago).
Extra spam prevention not enabled.

[Learn more about form handling in the docs](#)

Download as CSV
Delete form

Prevent spam

All form submissions are filtered for spam using Akismet. For additional protection, you can add a honeypot field and/or an explicit reCAPTCHA 2 challenge.

[Learn more](#)

Verified submissions

☐ Select all on page

Delete submission
Mark as spam
Collapse all

☒	팔달관 adas Restaurant Name 팔달관 Category 한식 Location 아주대 팔달관 Price Range 12313 Recommended Menu adas Review adas Submitter Name 오현석 Submitter Email ohs108@ajou.ac.kr Received yesterday at 11:20 AM from 39.117.36.84	Sep 14 (16 hours ago)
-------------------------------------	--	-----------------------

실습 개요

React Framework를 이용한 웹사이트 제작과 Netlify 배포

학습 목표

- React Framework를 이용한 웹 프론트엔드 개발 환경 구축
- JSX 문법을 이용한 React 컴포넌트 개발
- Git/GitHub를 활용한 버전 관리 및 자동 배포
- Netlify Forms을 활용한 서버리스 개발 체험

요구사항

- GitHub 저장소에 프로젝트 버전 관리가 바르게 되고 있는가?
- Netlify에 GitHub 저장소가 연결되어 자동 배포가 이루어 지고 있는가?
- Netlify 배포한 웹서비스에서 제시한 프로젝트 코드가 모두 바르게 동작하는가?

참고 : <https://github.com/ajou-hyunseok-oh/pwd-week3>

마감일

- 9월 21일 11시 59분 까지

GitHub URL

- (예: <https://github.com/username/pwd-week3>)

Netlify App URL

- (예: [https://pwd-week3-\[username\].netlify.app](https://pwd-week3-[username].netlify.app))

Q&A